

Processamento de Dados - Investimentos Públicos por Função (Sefaz-CE)

Paulo Icaro

Objetivo e Estrutura dos Dados

A rotina desenvolvida visa realizar a devida padronização nos dados de investimentos por função que municiam os modelos trabalhados no projeto.

A imagem a seguir representa a estrutura dos dados disponibilizados em cada uma das planilhas baixadas na página do [SIOF](#). Cada planilha representa um **Tipo** de investimento: Corrente, Pessoal, Equipamentos, Obras e Total. Em termos de informações efetivas, cada planilha contém oito campos. Destes, as colunas interesse são:

- Código
- Descrição
- Empenhado
- Pago

No campo Código, os investimentos estão classificados em Função, Subfunção, Programa e Projeto/Atividade. Para fins da pesquisa, apenas a primeira classificação interessa.

Levando em conta os pontos mencionados e que os dados são cumulativos, o objetivo dessa rotina é coletar as informações referente a Função considerando somente os valores de investimentos Empenhado e Pago. Os tópicos a seguir detalham cada etapa da rotina.



ESTADO DO CEARÁ EXECUÇÃO ORÇAMENTÁRIA - 2025

LEI Nº 19.154, de 30/12/2024
Consolidado por Função, Subfunção, Programa e PA

Acumulado até: ABRIL
PERCENTUAL S/ AUTORIZADO

Código	Descrição	Lei	Lei + Cred.	Empenhado	Pago	% Emp.	% Pago
01	LEGISLATIVA	733.513.646,00	733.512.431,68	212.433.032,98	199.132.981,72	28,96	27,15
031	AÇÃO LEGISLATIVA	31.500.782,00	31.500.782,00	8.068.982,95	7.600.136,81	25,62	24,13
435	DESENVOLVIMENTO DO PROCESSO LEGISLATIVO	31.500.782,00	31.500.782,00	8.068.982,95	7.600.136,81	25,62	24,13
20821	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS (FOLHA NORMAL) - ALECE	31.480.782,00	31.480.782,00	8.068.982,95	7.600.136,81	25,63	24,14
20822	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS (FOLHA COMPLEMENTAR) - ALECE	20.000,00	20.000,00	0,00	0,00	0,00	0,00
032	CONTROLE EXTERNO	240.488.089,00	240.486.874,68	66.793.780,39	63.570.204,78	27,77	26,43
421	GESTÃO ADMINISTRATIVA DO CEARÁ	240.488.089,00	240.486.874,68	66.793.780,39	63.570.204,78	27,77	26,43
20002	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS - FOLHA NORMAL - TCE	237.840.520,00	237.840.520,00	66.744.809,57	63.521.233,96	28,06	26,71
20371	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS - FOLHA COMPLEMENTAR - TCE	2.647.569,00	2.646.354,68	48.970,82	48.970,82	1,85	1,85
122	ADMINISTRAÇÃO GERAL	461.524.775,00	461.524.775,00	137.570.269,64	127.962.640,13	29,81	27,73
421	GESTÃO ADMINISTRATIVA DO CEARÁ	461.524.775,00	461.524.775,00	137.570.269,64	127.962.640,13	29,81	27,73
20006	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS (FOLHA NORMAL) - AL	461.024.775,00	461.024.775,00	137.358.875,75	127.795.773,71	29,79	27,72
20369	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS (FOLHA COMPLEMENTAR) - AL	500.000,00	500.000,00	211.393,89	166.866,42	42,28	33,37
02	JUDICIÁRIA	1.399.392.447,00	1.336.874.874,00	415.669.974,67	392.152.196,47	31,09	29,33
061	AÇÃO JUDICIÁRIA	1.060.577.293,00	998.401.276,76	310.193.791,51	292.209.659,39	31,07	29,27
192	EXCELÊNCIA NO DESEMPENHO DA PRESTAÇÃO JURISDICIONAL	1.060.577.293,00	998.401.276,76	310.193.791,51	292.209.659,39	31,07	29,27
20185	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS - FOLHA NORMAL - TJ (1º GRAU).	1.054.126.522,00	991.950.505,76	308.657.708,87	290.678.639,31	31,12	29,30
20489	PAGAMENTO DE DESPESAS DE PESSOAL E ENCARGOS SOCIAIS - FOLHA COMPLEMENTAR - TJ (1º GRAU).	6.450.771,00	6.450.771,00	1.536.082,64	1.531.020,08	23,81	23,73

Bibliotecas e Arquivos na Pasta

Para execução dessa rotina, duas bibliotecas foram utilizadas:

- **pandas**: biblioteca para manipulação, limpeza e análise de dados.
- **os**: biblioteca padrão do python que permite interagir com o sistema operacional.

Importadas as devidas bibliotecas, os arquivos que serão trabalhados são devidamente mapeados por meio da função **listdir** da biblioteca **os**. Nessa etapa também é criado um *dataframe*, **dataset_full**, que irá armazenar todo o conjunto de dados final.

```
# ===== #  
# === Bibliotecas === #  
# ===== #  
import pandas as pd  
import os  
  
# ===== #  
# === Arquivos de Dados === #  
# ===== #  
folder_files = os.listdir('Dataset/Investimentos_Funcao/')
```

```
dataset_full = pd.DataFrame()

# --- Prints --- #
print(*folder_files[0:10], sep = '\n')
```

```
F_JAN_2016_PESSOA.XLS
F_MAI_2018_CORRE.XLS
F_ABR_2023_CORRE.XLS
F_JUL_2020_CORRE.XLS
F_FEV_2015_CORRE.XLS
F_DEZ_2023_TOTAL.XLS
F_FEV_2022_TOTAL.XLS
F_NOV_2025_EQUIP.XLS
F_AGO_2022_OBRAS.XLS
F_NOV_2018_TOTAL.XLS
```

Processamento de Dados

Nesta etapa, uma estrutura *loop* é utilizada realizar a manipulação da rotina, onde cada etapa do tratamento é executado em cada conjunto de informações contidos nas planilhas.

Na leitura dos arquivos, somente as colunas de interesse são coletadas. Além disso, toda e qualquer linha nula é removida desse conjunto. Esse conjunto de dados remodelado recebe o nome de **dataset**.

Ao **dataset** são acrescidos quatro novos campos representando, respectivamente, i) período, ii) tipo de investimento, iii) ano, e iv) mês. Tais informações são extraídas justamente do nome de cada arquivo processado¹.

Após esses tratamentos iniciais, algumas linhas de comando são responsáveis por duas partes cruciais do procedimento. Primeiramente, são identificas e filtradas somentes as linhas referente a cujo código corresponde a Função. Dado a estrutura de looping, a segunda parte diz respeito a regra de acumulação das informações. **Cada planilha que passa pelo procedimento representa um dataset que contribui para um dataframe final chamado dataset_full contendo todas as informações por ano, mês e tipo de investimento e função.**

```
# ===== #
# === Processamento de Dados === #
# ===== #
for x in range(len(folder_files)):
```

¹Tal fato só é possível devido a padronização adotada para o nome dos arquivos. Para distinguir dos investimentos por programa, os arquivos recebem o prefixo **F_**. Por exemplo, para os investimentos em Equipamentos no período de Março de 2020, teria-se **F_MAR_2020_EQUIP.xlsx**.

```

# --- Leitura de arquivos --- #
dataset = pd.read_excel(io = 'Dataset/Investimentos_Funcao/' + folder_files[x],
                        header = 10,
                        usecols= 'C, F, K, N',
                        dtype = {'Código':str})

# --- Removendo valores vazios --- #
dataset = dataset.dropna()

# --- Adicionando colunas --- #
dataset = dataset.assign(periodo = folder_files[x][6:10] + '/' +
                        folder_files[x][2:5],
                        tipo = folder_files[x][11:16],
                        ano = folder_files[x][6:10],
                        mes = folder_files[x][2:5])

# --- Algumas substituições --- #
replacements = {'JAN':'01', 'FEV':'02', 'MAR':'03', 'ABR':'04', 'MAI':'05',
                'JUN':'06', 'JUL':'07', 'AGO':'08', 'SET':'09', 'OUT':'10', 'NOV':'11',
                'DEZ':'12'}
for old, new in replacements.items():
    dataset['mes'] = dataset['mes'].replace(old,new)

# --- Identificando linhas correspondentes a Função --- #
function_flag = dataset['Código'].str.len() == 2

# --- Selecionando somente as linhas que se encaixam na condição anterior --- #
dataset = dataset[function_flag]

# --- Reordenando e renomeando --- #
dataset = dataset.reindex(columns = ['periodo', 'ano', 'mes', 'tipo', 'Código',
                                    'Descrição', 'Empenhado', 'Pago'])
dataset.rename(columns = {'Descrição':'funcao', 'Código':'codigo',
                          'Empenhado':'empenhado', 'Pago':'pago'}, inplace = True)

# --- Empilhando dataset's no dataset_full --- #
if x == 0:
    dataset_full = dataset
else:
    dataset_full = pd.concat([dataset_full, dataset])
print(dataset_full.head(10))

```

	periodo	ano	mes	tipo	codigo	funcao \
0	2016/JAN	2016	01	PESSO	04	PESSOAL E ENCARGOS SOCIAIS
0	2018/MAI	2018	05	CORRE	01	LEGISLATIVA
61	2018/MAI	2018	05	CORRE	02	JUDICIÁRIA

135	2018/MAI	2018	05	CORRE	03	ESSENCIAL À JUSTIÇA
238	2018/MAI	2018	05	CORRE	04	ADMINISTRAÇÃO
556	2018/MAI	2018	05	CORRE	06	SEGURANÇA PÚBLICA
865	2018/MAI	2018	05	CORRE	08	ASSISTÊNCIA SOCIAL
1015	2018/MAI	2018	05	CORRE	09	PREVIDÊNCIA SOCIAL
1124	2018/MAI	2018	05	CORRE	10	SAÚDE
1519	2018/MAI	2018	05	CORRE	11	TRABALHO

	empenhado	pago
0	7.130100e+08	7.063338e+08
0	2.260264e+08	2.140815e+08
61	4.248100e+08	4.128724e+08
135	1.723213e+08	1.678611e+08
238	3.674781e+08	3.499799e+08
556	8.496340e+08	8.186370e+08
865	7.591048e+07	7.330846e+07
1015	1.271395e+09	1.271218e+09
1124	1.212032e+09	1.081297e+09
1519	1.634733e+07	1.576241e+07

Ajuste para Dados Cumulativos

Nessa seção da rotina, o objetivo principal é chegar ao investimento efetivo que se deu em determinado período. Dessa forma, a ideia é sempre tomar a diferença entre os períodos adotando a seguinte regra:

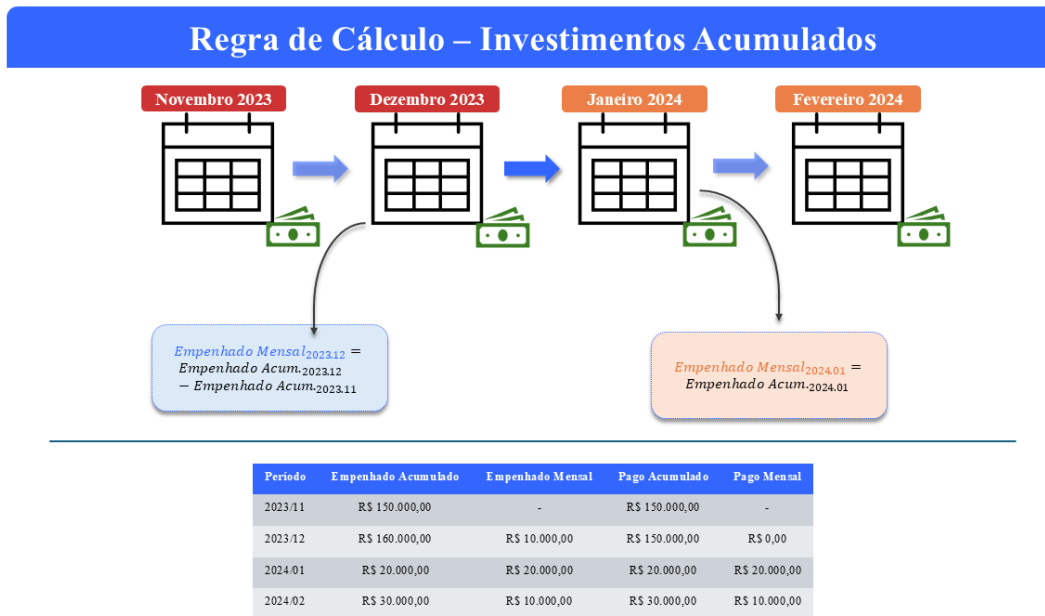
$$invest_t = invest_acum_t - invest_acum_{t-1} \quad (t \neq 1)$$

É interessante ressaltar que para chegar nesse cálculo o dataframe **dataset_full** precisa estar devidamente ordenado, evitando assim variações entre períodos, tipos de investimento e mesmo funções diferentes. Em vista dessa necessidade, a função **sort_values** desempenha esse papel de ordenamento, onde a seguinte ordem é adotada:

- Função
- Tipo
- Ano
- Mês

No **dataset_full** duas novas colunas representando o investimento mensal, **empenhado_mensal** e **pago_mensal**, são adicionadas, ao passo que os campos **empenhado** e **pago** passam a se chamar **empenhado_acumulado** e **pago_acumulado**.

Para valores que representam o mês de janeiro, uma regra em looping foi gerada. Uma vez que os dados estão devidamente ordenados e não há possibilidade alguma de conflito de informações, ao comparar a linha atual com a anterior, se a diferença entre os meses for superior a 1, então tem-se a confirmação de um cenário onde a linha atual representa janeiro e a linha anterior representa dezembro do ano anterior. Nesse caso, a rotina entende que o valor mensal é o mesmo valor acumulado. Nos demais cenários a regra padrão é adotada. O infográfico a seguir ajuda a compreender a regra adotada.



```
# ===== #
# === Ajustes para Dados Cumulativos === #
# ===== #

# --- Ordenamento --- #
dataset_full = dataset_full.sort_values(by = ['funcao', 'tipo', 'ano',
↪ 'mes']).reset_index(drop = True)

# --- Ajuste nos dados cumulativos --- #
dataset_full['empenhado_mensal'] = dataset_full['empenhado'] -
↪ dataset_full['empenhado'].shift(1)      # Inserting adjusted values
dataset_full['pago_mensal'] = dataset_full['pago'] - dataset_full['pago'].shift(1)
↪ # Inserting adjusted values

# --- Looping de ajuste para valores com datas truncadas --- #
for i in range(len(dataset_full)):
    if i == 0:
```

```

dataset_full.loc[0, 'empenhado_mensal'] = dataset_full.loc[0, 'empenhado']
dataset_full.loc[0, 'pago_mensal'] = dataset_full.loc[0, 'pago']
if i != 0 and int(dataset_full.loc[i, 'mes']) - int(dataset_full.loc[i-1, 'mes'])
↪ != 1:
    dataset_full.loc[i, 'empenhado_mensal'] = dataset_full.loc[i, 'empenhado']
    dataset_full.loc[i, 'pago_mensal'] = dataset_full.loc[i, 'pago']

# --- Renomeando colunas --- #
dataset_full.rename(columns = {'empenhado':'empenhado_acumulado',
↪ 'pago':'pago_acumulado'}, inplace = True)

# --- Prints --- #
print(dataset_full.head(10))

```

	periodo	ano	mes	tipo	codigo	funcao	empenhado_acumulado \
0	2015/JAN	2015	01	CORRE	04	ADMINISTRAÇÃO	4.575043e+07
1	2015/FEV	2015	02	CORRE	04	ADMINISTRAÇÃO	1.014506e+08
2	2015/MAR	2015	03	CORRE	04	ADMINISTRAÇÃO	1.618377e+08
3	2015/ABR	2015	04	CORRE	04	ADMINISTRAÇÃO	2.249567e+08
4	2015/MAI	2015	05	CORRE	04	ADMINISTRAÇÃO	3.140338e+08
5	2015/JUN	2015	06	CORRE	04	ADMINISTRAÇÃO	3.770741e+08
6	2015/JUL	2015	07	CORRE	04	ADMINISTRAÇÃO	4.692636e+08
7	2015/AGO	2015	08	CORRE	04	ADMINISTRAÇÃO	5.351476e+08
8	2015/SET	2015	09	CORRE	04	ADMINISTRAÇÃO	6.108187e+08
9	2015/OUT	2015	10	CORRE	04	ADMINISTRAÇÃO	6.842705e+08

	pago_acumulado	empenhado_mensal	pago_mensal
0	4.523944e+07	45750426.81	45239439.15
1	9.883464e+07	55700184.07	53595205.38
2	1.565762e+08	60387091.81	57741507.67
3	2.166955e+08	63119031.42	60119361.16
4	2.837923e+08	89077105.25	67096791.08
5	3.498681e+08	63040279.38	66075759.10
6	4.413997e+08	92189448.18	91531595.12
7	5.069026e+08	65884006.12	65502916.50
8	5.811869e+08	75671120.30	74284283.18
9	6.595495e+08	73451782.01	78362653.39

Armazenamento dos Resultados

Após todo o tratamento aplicado aos dados, a base final, **dataset_full**, recebe um último tratamento antes dos dados serem finalmente armazenados em um arquivo excel (.xlsx).

A base de dados final que até então continha 10 colunas passa por uma espécie de “verticalização”² das informações. As colunas de valores agora são divididas em duas colunas. Enquanto os valores em si se alinham sob uma única coluna nomeada **valor**, uma coluna nomeada **categoria** é gerada para representar a categoria de investimento que está sendo tratada. Os campos período, ano, mês, tipo, código e função permanecem como variáveis categóricas.

Ao final da rotina é realizada uma limpeza no *enviroment* mantendo somente o dataframe final para visualização.

```
# ===== #
# === Armazenamento dos Resultados === #
# ===== #

# --- Ajuste vertical na estrutura dos dados --- #
dataset_full = dataset_full.melt(
    id_vars = ['período', 'ano', 'mes', 'tipo', 'código', 'função'],
    value_vars = ['empenhado_acumulado', 'pago_acumulado', 'empenhado_mensal',
↪ 'pago_mensal'],
    var_name = 'categoria',
    value_name = 'valor'
)
print(dataset_full.head(10))
```

	período	ano	mes	tipo	código	função	categoria \
0	2015/JAN	2015	01	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
1	2015/FEV	2015	02	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
2	2015/MAR	2015	03	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
3	2015/ABR	2015	04	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
4	2015/MAI	2015	05	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
5	2015/JUN	2015	06	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
6	2015/JUL	2015	07	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
7	2015/AGO	2015	08	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
8	2015/SET	2015	09	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado
9	2015/OUT	2015	10	CORRE	04	ADMINISTRAÇÃO	empenhado_acumulado

	valor
0	4.575043e+07
1	1.014506e+08
2	1.618377e+08
3	2.249567e+08
4	3.140338e+08
5	3.770741e+08

²Esse procedimento é realizado por meio da função [melt](#).


```
6 4.692636e+08
7 5.351476e+08
8 6.108187e+08
9 6.842705e+08
```

```
# --- Armazenando --- #
with pd.ExcelWriter(path = 'investimentos_siof_ceara_funcao.xlsx', engine =
↳ 'xlsxwriter') as writer:
    dataset_full.to_excel(excel_writer = writer, sheet_name = 'investimentos_funcao',
↳ index = False)

    # Formatação básica
    workbook = writer.book
    worksheet = writer.sheets['investimentos_funcao']
    money_formatting = workbook.add_format({'num_format': 'R$#,##0'})
    #perc_formatting = workbook.add_format({'num_format': '0.0%'})
    worksheet.set_column('H:H', 15, money_formatting)
    #worksheet.set_column('K:L', 15, perc_formatting)

# --- Limpeza --- #
del(dataset, folder_files, money_formatting, perc_formatting, workbook, worksheet,
↳ writer, x)
```